IY3660 – Applications of Cryptography

Week 10 Lecture
Digital Signatures

Elizabeth Quaglia

Information Security Group

ROYAL HOLLOWAY UNIVERSITY OF LONDON

In today's lecture

- Introduce digital signature schemes
- Security of signatures
- Examples of signatures schemes
  - RSA-based
  - Schnorr
  - DSA

Digital signatures

Introduction to digital signatures

- Assume Alice wants to send a message *m* to Bob
- The adversary controls the network, as usual
- The adversary would like to compromise the **integrity** of message *m*
  - The adversary wants Bob to receive and accept an alternative message *m'*
- Alice and Bob need a cryptographic mechanism that detects modifications
- Existing solution to this problem?
  - MAC algorithm
- What if Bob and Alice do not share a symmetric key?

Signatures: signing

- (Digital) **signature schemes** are public-key analogue of MAC algorithms
- Alice has a signing key *sk* and a verification key *vk*
- Bob is assumed to have an ***authentic*** copy of *vk*
- There are methods to distribute and authenticate verification keys
- Alice uses *Sign* algorithm to compute signatures on messages *m:*

$$\sigma = Sign(sk,m)$$

- Alice sends (m, σ) to Bob in place of m

Signatures: verification

- Bob uses *Vrf* algorithm and verification key *vk* to verify the signature
- *Vrf* algorithm outputs 0 or 1
- Bob accepts that *m* comes from Alice if $Vrf(vk,m,\sigma) = 1$ and rejects *m* otherwise
- It must be hard for the adversary to find pairs *m* and σ such that $Vrf(vk,m,\sigma) = 1$
  - It should be hard to forge signatures that verify using Alice's verification key

<u>NB:</u> our formulation of signature schemes requires *m* as an input to *Vrf*

- Some schemes provide message recovery
  - They can recover *m* (or part of *m)* from the signature σ during verification

Formal definition of signature scheme

Definition:

A signature scheme consists of a triple of algorithms (**KGen**, **Sign**, **Vrf**)

KGen is a randomised algorithm that outputs key pairs (vk,sk)

Sign takes as input sk and a message $m \in \{0, 1\}^*$, and outputs a signature σ

Vrf takes as input a triple (vk,m,σ) and outputs 0 or 1

Correctness:

For all key pairs (vk,sk) output by **KGen**, for all $m \in \{0, 1\}^*$,

if σ = **Sign**(sk,m) then **Vrf**(vk,m,σ) = 1

NB both **Sign** and **Vrf** may be randomised algorithms

Digital signatures, handwritten signatures and non-repudiation

- In 1976 Diffie and Hellman suggested use of public-key digital signatures may play similar role to handwritten signatures as legal evidence

" In current business, the validity of contracts is guaranteed by signatures. A signed contract serves as legal evidence of an agreement which the holder can present in court if necessary."

"*[u]nforgeable digital signatures are needed*" to protect against a message being "*later repudiated by either the transmitter or sender.*"

Questions:

- What is the relationship between handwritten and digital signatures?
- To what extent do digital signatures provide non-repudiation?

Digital signatures vs handwritten signatures

- **Environment differences** (e.g., form, creation, availability, etc)
- **Security differences** (e.g., consistency, uniqueness, ease of forgery, etc)
- **Practical differences** (e.g., cost, acceptability, legal recognition, etc)
- **Flexibility differences** (e.g., binding to data, advanced signatures, etc)

Non-repudiation

- **Non-repudiation**: inability to disavow a previous agreement
- One of the four security services that cryptography is claimed to provide
- How good are (digital) signatures for providing non-repudiation?
- Is a digital signature admissible legally?

EU Directive on Community Framework for Electronic Signatures (1999/93/EC)

- Legal framework for use of electronic signatures
- Technology neutral
  - Not focused solely on signatures based on public key cryptography

1999/93/EC: signatures definitions

- **Electronic signature***:* data in electronic form, which are attached to or logically associated with other electronic data, and which serve as a method of authentication
- **Advanced electronic signature***:* electronic signature, which:
  - Is uniquely linked to the signatory
  - Is capable of identifying the signatory
  - Is created using means that the signatory can maintain under their sole control
  - Is linked to the data to which it relates in such a manner that any subsequent change of the data is detectable

- Do (PK) digital signatures meet these conditions?

ref: "Legal Aspects of Digital Signatures and Non-Repudiation", Ilja Ponka (2000)
https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.27.1602&rep=rep1&type=pdf

1999/93/EC: legal admissibility of signatures

Article 5 of the Directive states that **advanced electronic signatures**, which are

- Based on a qualified certificate
- Created by secure-signature-creation device

both

- Satisfy the legal requirements of a signature for electronic data in the same way as a hand-written signature for paper-based data
- Are admissible as evidence in legal proceedings

ref: "Legal Aspects of Digital Signatures and Non-Repudiation", Ilja Ponka (2000)
https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.27.1602&rep=rep1&type=pdf

Standards for signature schemes

NIST

- DSA, ECDSA, RSA-PSS, PKCS#1 v1.5 with RSA

NSA Suite B

- ECDSA

NESSIE

- ECDSA, RSA-PSS, ~~SFLASH~~ (broken)

CRYPTREC

- DSA, ECDSA, RSA-PSS, PKCS#1 v1.5 with RSA

IEEE P1363

- DSA, ECDSA, PSS w/ RSA or Rabin-Williams, PKCS#1 v1.5 w/ RSA or Rabin-Williams
- Some signature schemes with message recovery

NIST Post-Quantum Cryptography standardization process (ongoing)

- Dilithium, Falcon, Sphincs+
- More still under consideration (SQIsign, Hawk..)

Applications of signatures

- Public verification of message authenticity/integrity
- Code-signing
- Entity authentication and identification
  - Sign challenge messages to prove possession of *sk* matching a specific *vk*
  - Used as a building block in more complex protocols
- Certification systems, public key infrastructures (PKI)
  - Use signatures to authenticate other signing keys and public encryption keys
- Cryptocurrencies and blockchain applications
  - Public verification of transactions (authenticity)

Can digital signatures be widely deployed?

- Ideally, cryptographic signatures could replace handwritten signatures
- Deployments exist as part of electronic identity cards in some countries, e.g., Belgium
- Required legal frameworks are in place worldwide
- Practical challenges remain:
  - Requirements for tamperproof hardware (e.g., smartcards)
  - Requirements for special terminals to interact with it
  - Management of keys via suitable infrastructure
  - Usability of software especially for non-expert users
  - Avoiding security pitfalls, e.g., bad randomness

Security of signatures

What security do you expect?

- Do we expect to protect the confidentiality of m?
- Do we expect to protect the integrity of m?
- Can Alice later deny having created signature σ on a message m if $\mathrm{Vfy}(vk,m,\sigma) = 1$?
- What happens if an attacker can derive sk from vk?
- Can the attacker get hold of signatures σ_i for some messages m_i?
- Can the attacker even choose these messages?
- How would we model that capability?

Formal definition of security for signatures

- Security for a signature scheme (**KGen**, **Sign**, **Vrf**) is formalised in terms of a security game between a challenger and an adversary
- Challenger generates a key pair (vk,sk) by running **KGen**
- Challenger gives vk to adversary
- Adversary runs, with access to a signing oracle: adversary sends m and gets back σ = **Sign**(sk,m)
- No access to a verification oracle, cf. MAC security
- Adversary finally outputs (m^*, σ^*)
- Winning condition:

  Adversary wins if m^* is distinct from all the m queried to the signing oracle AND if Vfy(vk,m^*,σ^*) = 1

Formal definition of security for signatures

Winning condition:

Adversary wins if it can generate (m^*, σ^*), with m^* distinct from any m queried to the signing oracle, such that $\mathrm{Vfy}(vk, m^*, \sigma^*) = 1$

Scheme (**KGen**, **Sign**, **Vrf**) is said to be (E)UF-CMA secure if no adversary consuming reasonable resources wins with significant probability

- (E)UF = (existentially) unforgeable
- CMA = chosen message attacks

This can be formalised, but we won't do so here

Consequences of the definition

UF-CMA security implies:

- It's hard for the adversary to find *sk* from *vk*: why?
- It's hard for the adversary to create a forgery given just *vk*: why?
- It's hard for the adversary to create a forgery when given access to signatures on random messages (that the adversary does not control): why?
- The legitimate owner of the key pair (*vk*,*sk*) cannot deny having created a signature σ on a message m: why?

Strong unforgeability

Strong unforgeability (SUF-CMA security):

Adversary wins if (m^*,σ^*) is distinct from all the pairs (m,σ) appearing in the security game AND if $\mathrm{Vfy}(vk,m^*,\sigma^*) = 1$

- Adversary could now win by producing a new signature σ^* on a message m that was previously queried to the signing oracle
- It is easier for the adv. to win, so SUF-CMA is **stronger** than UF-CMA
- Moreover, if scheme is SUF-CMA, then it is also UF-CMA
- The converse is not true: a scheme may be UF-CMA and not SUF-CMA

Examples of signatures: RSA setting

RSA-based signatures

First attempt: **naive** RSA-based signatures:

- KGen: as in RSA encryption, we set $vk = (N,e)$ and $sk = d$, where $N = pq$ is a product of two large primes and $ed = 1 \bmod (p-1)(q-1)$
- Sign: we use the "private key" $sk = d$ to sign:

$$\sigma = m^d \bmod N$$

- Vrf: given (m,σ), we check whether $\sigma^e = m \bmod N$
  If yes, return 1; otherwise return 0

DO NOT USE!

Security?

- Given signatures σ_1 and σ_2 on messages m_1 and m_2, simple algebra shows that $\sigma_1\sigma_2 \bmod N$ is a valid signature on m_1m_2!
- This makes creation of forgeries trivial
- Exercise: formulate the attack in the context of the UF-CMA security game

Hash-based RSA signatures

Second attempt: hash-based RSA signatures:

- KGen: as in RSA encryption, we set $vk = (N,e)$ and $sk = d$, where $N = pq$ is a product of two large primes and $ed = 1 \bmod (p-1)(q-1)$
- Sign: we use the "private key" $sk = d$ to sign:

$$\sigma = \mathrm{H}(m)^d \bmod N,$$

  where H is a hash function
- Vrf: given (m,σ), we check whether $\sigma^e = \mathrm{H}(m) \bmod N$

  If yes, return 1; otherwise return 0

- Using H destroys the multiplicative structure that enabled previous attack
- Also allows signing of long messages
- This attempt follows the so-called hash-and-sign paradigm

Hash-based RSA signatures

Second attempt: hash-based RSA signatures:

- Sign: we use the "private key" $sk = d$ to sign:

$$\sigma = \mathrm{H}(m)^d \mod N,$$

  where H is a hash function

Security?

- What if we can find a hash collision? That is, a pair of messages m_1, m_2 such that $\mathrm{H}(m_1) = \mathrm{H}(m_2)$?
- Then

$$\sigma_1 = \mathrm{H}(m_1)^d = \mathrm{H}(m_2)^d = \sigma_2 \mod N$$

- Hence a signature σ_1 on m_1 is also a valid signature on m_2
- This is an issue for any signature scheme following the hash-and-sign paradigm!

Hash-based RSA signatures: birthday attack

- There is a generic attack against hash-based RSA signatures using n-bit hash functions, which relies on producing hash collisions
- Suppose you have a document *X* which Alice is happy to sign with her private signing key, e.g., a letter saying that you work at her company
- Now let *Y* be a malicious document which you would like Alice to sign, e.g., a letter giving you full control of the company
- Yuval (1979) proposed an attack in which you can obtain a signature of the malicious document with complexity of ~ $2^{n/2}$ hash operations (i.e., a birthday attack)

Details of the attack

1. Generate around $2^{n/2}$ variations X_i of the document X (e.g., extra spaces, empty padding before/after the document, etc)
2. Store X_i on a table T, indexed by the hash $H(X_i)$ for i = {1, …, $2^{n/2}$ }
3. Generate variations Y_j of the document Y (same as above), compute $H(Y_j)$, and check whether there is $[H(X_i)\ ,\ X_i]$ in T such that $H(Y_j) = H(X_i)$
4. By the birthday paradox, we should expect a match after around $2^{n/2}$ variations Y_j of the document Y
5. Then ask Alice to sign the document X_i with its hash-based RSA private signing key It follows that the signature is also valid for Y_j

- Generic attack against hash-based RSA signatures
- Complexity square-root on the size of the hash function

Hash-based RSA signatures

- Our hash function must be collision-resistant
- The generic attack just shown means that the maximum bit-security a hash-based RSA signature scheme can offer is the half of the length of the hash function output
- Regardless of the RSA parameters, e.g., 4096 bits, if we use e.g., SHA-1, the scheme can offer (at most) 80 bits of security
  - We saw that SHA-1 in not collision resistant anyway!
- Moreover, even if we use SHA-256, $H(m)$ will have 256 bits, and when considered as an integer in [0, N -1], we may run into trouble…
  - We saw that RSA encryption of small messages could be broken

Full-domain hash-based RSA signatures

Third attempt: full-domain hash RSA signatures:

- KGen: as in RSA encryption, we set $vk = (N,e)$ and $sk = d$, where $N = pq$ is a product of two large primes and $ed = 1 \bmod (p-1)(q-1)$
- Sign: we use the "private key" $sk = d$ to sign:

$$\sigma = \mathrm{H}(m)^d \bmod N,$$

  where H is a hash function *with outputs in* $\{0,1,\ldots, N-1\}$
- Vrf: given (m,σ), we check whether $\sigma^e = \mathrm{H}(m) \bmod N$

Security?

- If we regard H as being a random function, then we can prove that this scheme is UF-CMA secure under the assumption that RSA inversion is hard

Full-domain hash-based RSA signatures

RSA inversion problem: given (N,e) and random number x modulo N, compute $x^d \bmod N$ (where $de = 1 \bmod (p-1)(q-1)$)

- If factoring is easy, then RSA inversion is easy
- We have no proof that if factoring is hard, then RSA inversion is hard
  - But it is widely believed to be true

However:

- Security proof for only weakly relates UF-CMA security to RSA inversion
- Building hash functions that have outputs in $\{0,1,\ldots,N-1\}$ is inconvenient

Hash-based RSA signatures with padding

Fourth attempt: RSA signatures with padding:

- Sign: we use the "private key" $sk = d$ to sign:

$$\sigma = [\ \mathrm{pad}(\mathrm{H}(m))\ ]^d \bmod N,$$

  where H is a hash function and pad(.) is some deterministic padding scheme
- Vrf: given (m,σ), we check whether $\sigma^e = \mathrm{pad}(\mathrm{H}(m)) \bmod N$

- This scheme is widely standardised and still in wide use, e.g., PKCS#1 v.1.5 standard, ANSI X9.31, IEEE P1363a, SSL/TLS, IPsec, EMV
- **PKCS#1 v.1.5 padding**: 00 01 FF... FF 00 || c || H(m), with constant c

hash-based RSA signatures with padding

Security?

- No security proof for the scheme with PKCS#1 v.1.5 padding is known
- Signatures can be forged if the constant part of the padding is too short
- Padding check/removal often wrongly implemented

RSA-PSS

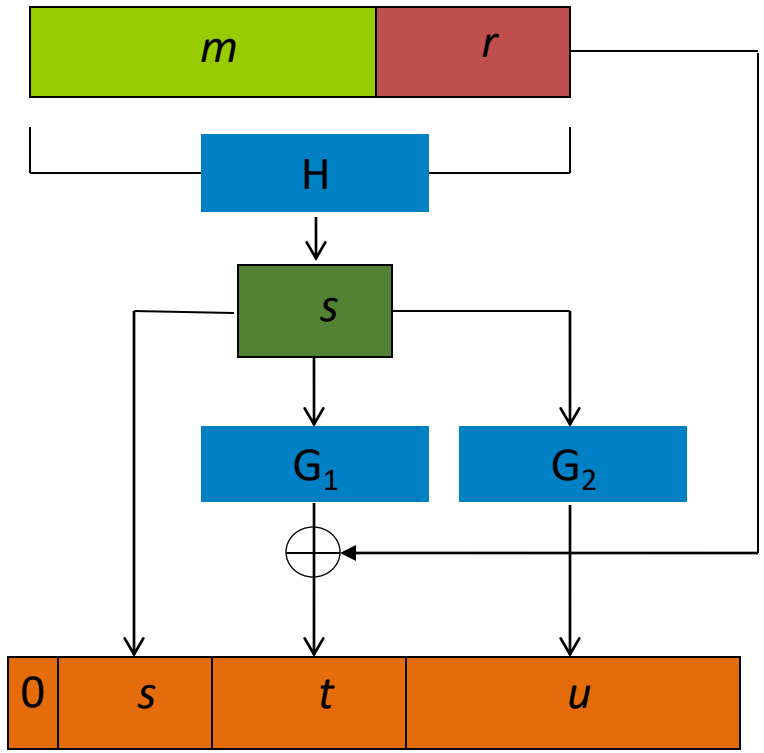
Fifth attempt: RSA Probabilistic Signature Scheme (RSA-PSS):

- KGen: as in RSA encryption, we set $vk = (N,e)$ and $sk = d$, where $N = pq$ is a product of two large primes and $ed = 1 \bmod (p-1)(q-1)$
- Sign (simplified):

1. Generate a random value r (with 256 bits, say)
2. Compute $s = H(m||r)$, $t = G_1(s) \oplus r$, $u = G_2(s)$ where G_1, G_2 and H are suitable hash functions (sum of their output lengths should be λ-1, where λ is bit-length of N)
3. Return as the signature:

$$\sigma = (0 \,||\, s \,||\, t \,||\, u)^d \bmod N$$

RSA-PSS (simplified) encoding



RSA-PSS verification

- Vrf (simplified): given (m,σ):

1. Compute σ^e mod N and parse the result as $b \,||\, s \,||\, t \,||\, u$ where b is a bit and s, t, u have the correct lengths (determined by output lengths of H, G_1, G_2)
2. Output "0" if $b = 1$
3. Compute r' by inverting the equations for s, t:
   - compute $r' = G_1(s) \oplus t$
4. Re-encode r' and m and check correctness:
   - Compute $s' = H(m||r')$, $t' = G_1(s') \oplus r'$, $u' = G_2(s')$
   - Check that $s' = s$ *AND* $u' = u$; output "0" if this fails
   - Otherwise output "1"

Security of RSA-PSS

- Assuming G_i and H behave like random functions, the UF-CMA security of RSA-PSS can be ***tightly*** related to the hardness of RSA inversion
- RSA-PSS can be instantiated with "ordinary" collision-resistant hash functions, e.g., SHA-256 (no hashing onto full domain is needed)
- RSA-PSS is standardised: PKCS#1 v2.1, IEEE P1363a-2004, IETF RFC 3447,…

If you must use RSA signatures, then RSA-PSS is the right choice of scheme

RSA-based signatures: summary

- Naïve RSA: DO NOT USE UNDER ANY CIRCUMSTANCES.
- Hash-then-sign: no known attack, but also no proof
  - This includes basic hash-then-sign, and PKCS#1 v.1.5 padding
- Full-Domain Hash: provably secure, with weak proof
- RSA-PSS: provably secure, with tight proof
  - If you want an RSA-based signature scheme, please only use RSA-PSS!

Examples of signatures: DL setting

Recall: the discrete logarithm problem (DLP)

- Let p, q be two prime numbers such that q divides $p-1$, and let g be an integer such that $g, g^2, g^3, \ldots g^q$ are distinct modulo p, and $g^q = 1 \bmod p$

- We say that the set $G_q = \{g, g^2, g^3, \ldots g^q\} = \{g^0, g^1, g^2, \ldots g^{q-1}\}$ of powers of g mod p is a cyclic group of prime order q with generator g

The discrete logarithm problem in G_q:

Given p, q, g and an element $y = g^x \bmod p$ from G_q,
where x is a random value in $\{0,1,\ldots,q-1\}$: find x

Signatures in the discrete logarithm setting

The discrete logarithm problem in G_q:

Given p, q, g and an element $y = g^x \bmod p$ from G_q,
for a random value x in {0,1,…,q-1}: find x

- **Recall**: we have an analogous problem over Elliptic Curves: ECDLP (using the additive cyclic group of points in an Elliptic Curve E)

- **Recall**: Diffie-Hellman key exchange and ElGamal encryption require a group where DLP is hard

- A natural question is: can we also use the DLP to build signature schemes?
- Yes – we will introduce Schnorr signatures and DSA

DL-based signatures: setup

- Say, we have p a 1024-bit prime, q a 160-bit prime, with q dividing $p-1$
- Let g be a generator of the subgroup of order q of the integers mod p.
- We can set the secret signing key x, where $1 \leq x \leq q-1$
- The corresponding verification key is $y = g^x \bmod p$
- We hash messages m before signing, so we also need hash function H

DL-based signatures: first attempt

- **First attempt:**
  - We produce a signature s ($0 \leq s \leq q-1$) for message m, and want it to be verified as

$$g^s == y^{H(m)} \bmod p \ ?$$

  - The signature can be computed with the private key: $s = x \,.\, H(m) \bmod q$

However, given s a signature of m, an attacker can recover the private key x as:

$$x = s \,.\, H(m)^{-1} \bmod q$$

- **Not secure!** Valid signatures leak the private key!

DL-based signatures: second attempt

- Again, signing key x, where $1 \leq x \leq q-1$, verification key $y = g^x \bmod p$
- **Second attempt:** hide x by masking it with a random value in the signature
- For each new signature we draw a fresh k, and make $h = g^k \bmod p$
- Then we make the signature as the pair (h, s), where

$$s = k + x \,.\, H(m) \bmod q$$

- Verification is to check whether

$$g^s == h \,.\, y^{H(m)} \bmod p \ ?$$

- Forgery is now easy: for any message m, an attacker can pick a random value s', compute $H(m)$, and make

$$h' = g^{s'} \,.\, y^{-H(m)} \bmod p$$

  - Then (h', s') is a valid signature for m! Why?
  - **Not secure!** Forgery is trivial!

DL-based signatures: third attempt

- **Third attempt:** we bind the fresh value $h = g^k \bmod p$ with the message
- Signature is the pair (h, s), where

$$s = k + x \,.\, H(m \,||\, h) \bmod q$$

  - Note: we have $1 \leq h \leq p\text{-}1$ and $1 \leq s \leq q\text{-}1$
- Verification is to check whether

$$g^s == h \,.\, y^{H(m \,||\, h)} \bmod p \;?$$

- Since h is used in the signature and in the hash, the previous attack (solving for h) does not seem to work
- One issue: size of the signature
  - h is same size of p (e.g., 1024 bits) and s same size of q (e.g., 160 bits)

Schnorr signatures

- Similar to the previous attempt, but uses the hash function in more places
- Signature σ is the pair (r, s), where

$$r = H(m \,||\, (g^s \,.\, y^{-r} \bmod p))$$

- Verification is, given $\sigma = (r, s)$, message m, and public key y, checking whether the relation above holds

To compute the signature of a message m, the signing party will:

- Generate at random a fresh k, and make $h = g^k \bmod p$
- Set $r = H (m \,||\, h)$
- Make $s = k + x.r \bmod q$

- Size of the signature is the size in bits of the output of H plus the size of q

Schnorr signatures: reuse of randomness

- Assume the signer reuses the value k *to* produce the signature of two different messages:

$$\sigma_1 = (r_1, s_1) \text{ for } m_1 \qquad \text{and} \qquad \sigma_2 = (r_2, s_2) \text{ for } m_2$$

- $h = g^k \bmod p$ will be the same
- $r_1 = H(m_1 \,||\, h)$ *and* $r_2 = H(m_2 \,||\, h)$ are expected to be different
- $s_1 = k + x.r_1 \bmod q$ and $s_2 = k + x.r_2 \bmod q$ are expected to be different

- The adversary can obtain two valid signatures (r_1, s_1) and (r_2, s_2), so can solve

$$s_1 - x.r_1 = s_2 - x.r_2 \bmod q$$

to recover the private key x !

Reuse of randomness is devastating for Schnorr signatures (and related schemes)!

Schnorr signatures: properties and use

- The Schnorr signature scheme
  - Is efficient
  - Produces short signatures
  - Can be proven secure based on
    - hardness assumption of DLP,
    - randomness assumptions of H
- Schnorr signatures have seen limit use
  - Initially subject to a patent (which only expired in 2008)
- There are however proposals to use it in some modern applications
  - E.g., it is now used to authenticate transactions in Bitcoin
- Edwards-curve Digital Signature Algorithm (EdDSA)
  - A variant of Schnorr signatures is used in several software applications

Digital Signature Algorithm (DSA)

- DSA was proposed by NIST in 1991
- Explicitly requires the use of a specific hash function
  - Namely, SHA-1
- Related to Schnorr, but more complicated
  - It is a variant of ElGamal signature scheme
  - Rationale: bypass Schnorr's patent
- Very different set of functional capabilities compared to RSA
  - Like Schnorr, DSA is a dedicated signature algorithm and cannot be readily converted into an encryption scheme.
- Defined over integers modulo a prime *p*
  - One can also define the elliptic curve version (ECDSA)

DSA set up and key generation

- System parameters require two primes p and q:
  - 160-bit prime q
  - 1024-bit prime p so that q divides $p-1$
  - Find g that generates G_q, as previously

- KGen:
  1. Select random signing key x, $1 \leq x \leq q-1$.
  2. Compute verification key $y = g^x \bmod p$.
  3. Output ($vk=y$, $sk=x$).

- The problem of finding signing key from verification key is the DLP
- Note: many users can share the same system parameters

DSA signing

- To sign message m
  1. Hash message m to get $H(m)$
  2. Generate random value k, $1 \leq k \leq q-1$
  3. Compute $r = (g^k \bmod p) \bmod q$
  4. Compute $k^{-1} \bmod q$
  5. Compute $s = k^{-1}(H(m) + x\, r) \bmod q$
  6. Output $\sigma = (r,s)$

- Signatures can be represented using 2 x 160 = 320 bits
  - DSA signatures are smaller than RSA signatures at same security level
- Signing requires one exponentiation mod p using an exponent k of 160 bits (a short exponent), plus inversion and multiplication mod q

DSA verification

- To verify that $\sigma = (r,s)$ is a signature for message m:
  1. Check that $1 \leq r \leq q-1$ and $1 \leq s \leq q-1$
  2. Compute $w = s^{-1} \bmod q$
  3. Compute $u_1 = w \,.\, \mathrm{H}(m) \bmod q$ and $u_2 = r \,.\, w \bmod q$
  4. Accept signature if the following equation holds:

$$(g^{u_1} y^{u_2} \bmod p) \bmod q = r$$

- Correctness: suppose $\sigma = (r,s)$ is a signature for message m. Then:

$$g^{u_1} y^{u_2} = g^{s^{-1}\mathrm{H}(m)} y^{r s^{-1}} = g^{s^{-1}(\mathrm{H}(m)+xr)} = g^{k} \bmod p$$

and so

$$(g^{u_1} y^{u_2} \bmod p) \bmod q = (g^{k} \bmod p) \bmod q = r$$

Security of DSA signature scheme

Concrete:

- Possible attacks:
  - Solving DLP mod p
  - $O(q^{1/2})$ attacks in the subgroup G_q of order q
- Need to choose p and q both large enough to prevent these attacks

Provable security:

- No clean security proof for DSA is known
- There are various proofs under different assumptions and heuristics

(In)security of DSA under randomness reuse

Example:

- Suppose the same random value k is used to sign two different messages, m_1 and m_2, giving signatures $\sigma_1 = (r_1, s_1)$ and $\sigma_2 = (r_2, s_2)$
- Then $r = (g^k \bmod p) \bmod q$ is the same in the two signatures
  - I.e., $r_1 = r_2 = r$ (say); so, this condition is detectable
- Moreover: $s_1 = k^{-1}(\mathrm{H}(m_1) + x\, r) \bmod q$ and $s_2 = k^{-1}(\mathrm{H}(m_2) + x\, r) \bmod q$
- So: $s_1 - s_2 = k^{-1}(\mathrm{H}(m_1) - \mathrm{H}(m_2)) \bmod q$
- This can be rearranged to yield: $k = (s_1 - s_2)^{-1}(\mathrm{H}(m_1) - \mathrm{H}(m_2)) \bmod q$
- We can recover k and then x, by solving
  $s_1 = k^{-1}(\mathrm{H}(m_1) + x\, r) \bmod q$ using the known values s_1, k, $\mathrm{H}(m_1)$, r

Can randomness failure arise in practice?

- The above key recovery issue in DSA and the related ECDSA scheme occurs regularly in practice!
- Openssl bug in Debian (2008):

"It was discovered that the RBG in Debian's openssl package is predictable. [...] It is strongly recommended that all cryptographic key material is recreated from scratch. [...] All DSA keys ever used on affected systems for signing should be considered compromised."

- Bad random number generator in Android allowed Bitcoins to be stolen (2013)
- Hackers recovered Sony's PlayStation 3 signing key (2010)
- The problem occurs naturally in virtualized environments

See: https://pages.cs.wisc.edu/~swift/papers/oakland14-rng.pdf

Hedging DSA against randomness failures

- Related attacks are possible if
  - some bits of the random values k can be predicted
  - weak generators are used to produce k
  - relations between the bits of k are known
- We can **hedge** against attacks by derandomising:
  - Generate k in signing using a pseudo-random function F with a key K: $k = F_K(vk \mid\mid m)$
  - Ensures different random(-looking) values for different vk and m
  - Need to keep PRF key K as part of signing key
- See RFC 6979 for related scheme ECDSA
- Technique applies more generally for randomised signature schemes

Homework

- Read Chapter 7 of "*Everyday Cryptography*"
- Read Chapter 10 of "*Serious Cryptography*"